

Creating Scatter Plots

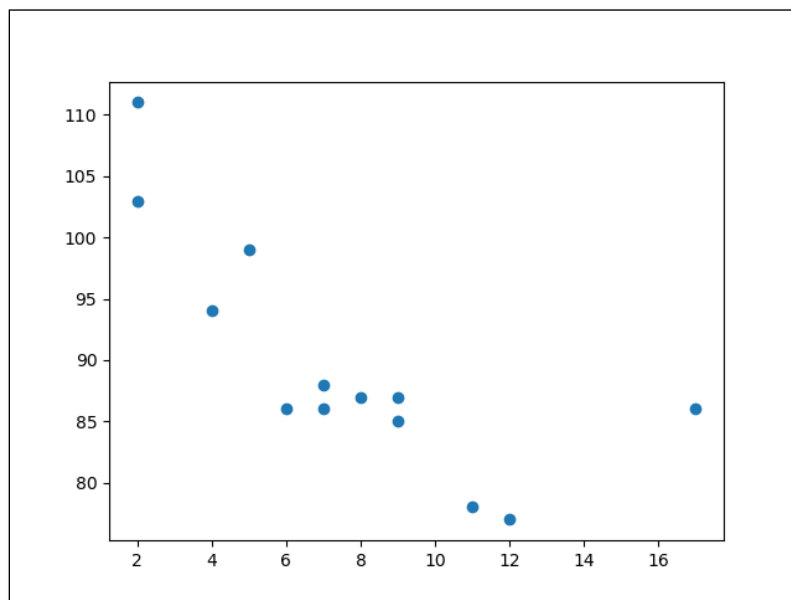
With Pyplot, you can use the **scatter()** function to draw a scatter plot.

The `scatter()` function plots one dot for each observation. It needs two arrays of the same length, one for the values of the x-axis, and one for values on the y-axis:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x, y)
plt.show()
```



The observation in the example above is the result of 13 cars passing by.

The X-axis shows how old the car is.

The Y-axis shows the speed of the car when it passes.

Are there any relationships between the observations?

It seems that the newer the car, the faster it drives, but that could be a coincidence, after all we only registered 13 cars.

Compare Plots

In the example above, there seems to be a relationship between speed and age, but what if we plot the observations from another day as well? Will the scatter plot tell us something else?

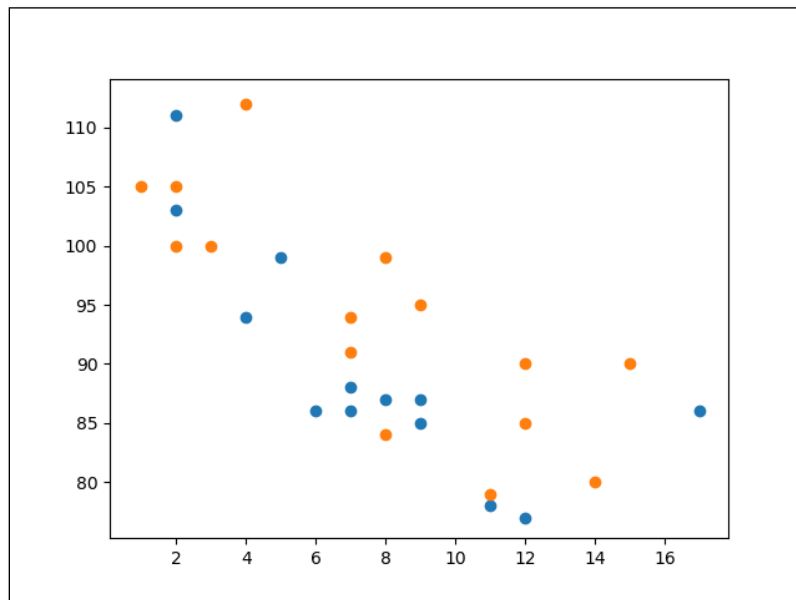
```
import matplotlib.pyplot as plt
import numpy as np

#day one, the age and speed of 13 cars:
x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
plt.scatter(x, y)

#day two, the age and speed of 15 cars:
```

```
x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
y =
np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85]
)
plt.scatter(x, y)

plt.show()
```



By comparing the two plots, I think it is safe to say that they both gives us the same conclusion: the newer the car, the faster it drives.

Colors

You can set your own color for each scatter plot with the color or the c argument:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
```

```
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
```

```
plt.scatter(x, y, color = 'hotpink')
```

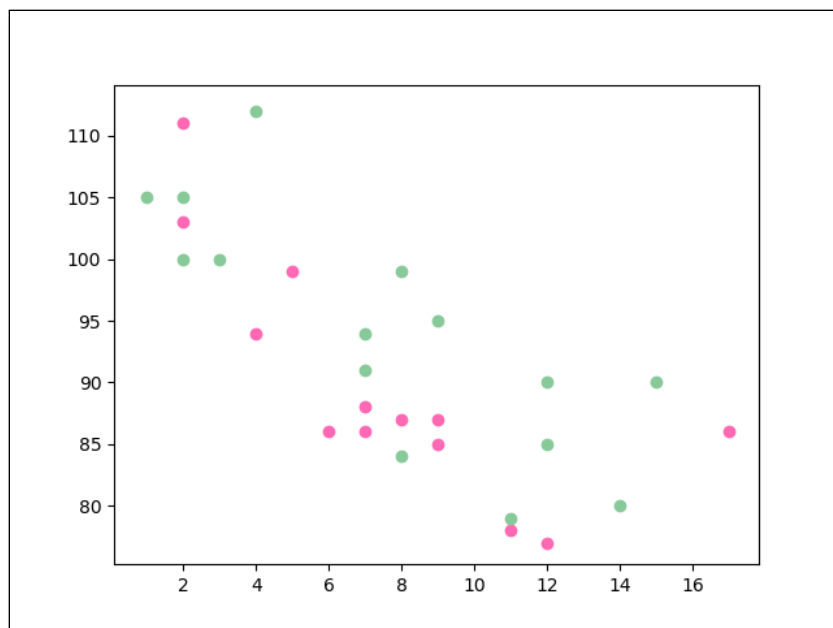
```
x = np.array([2,2,8,1,15,8,12,9,7,3,11,4,7,14,12])
```

```
y =
```

```
np.array([100,105,84,105,90,99,90,95,94,100,79,112,91,80,85])
```

```
plt.scatter(x, y, color = '#88c999')
```

```
plt.show()
```

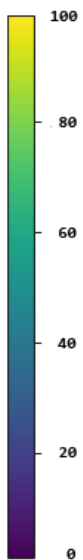


ColorMap

The Matplotlib module has a number of available colormaps.

A colormap is like a list of colors, where each color has a value that ranges from 0 to 100.

Here is an example of a colormap:



This colormap is called 'viridis' and as you can see it ranges from 0, which is a purple color, up to 100, which is a yellow color.

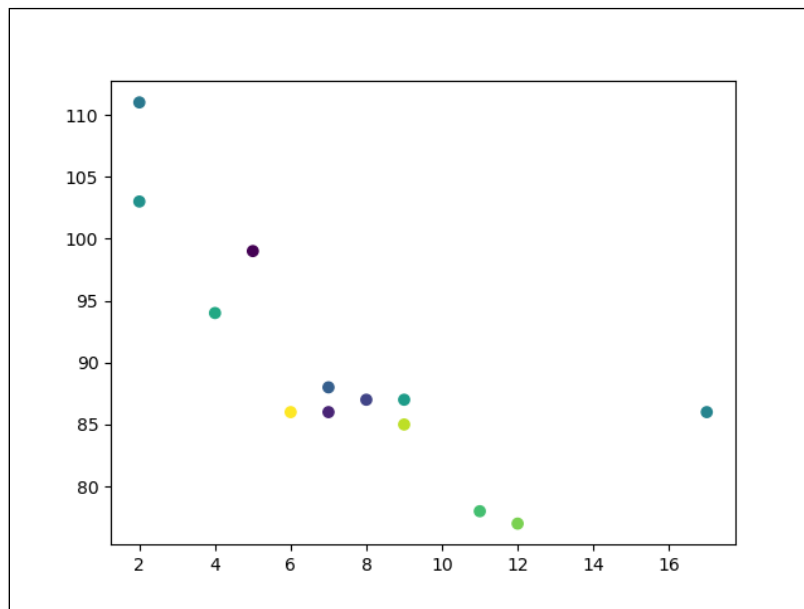
you can specify the colormap with the keyword argument **cmap** with the value of the colormap, in this case 'viridis' which is one of the built-in colormaps available in Matplotlib.

In addition you have to create an array with values (from 0 to 100), one value for each point in the scatter plot:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
colors = np.array([0, 10, 20, 30, 40, 45, 50, 55, 60, 70,
80, 90, 100])

plt.scatter(x, y, c=colors, cmap='viridis')
plt.show()
```



You can include the colormap in the drawing by including the `plt.colorbar()` statement:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
```

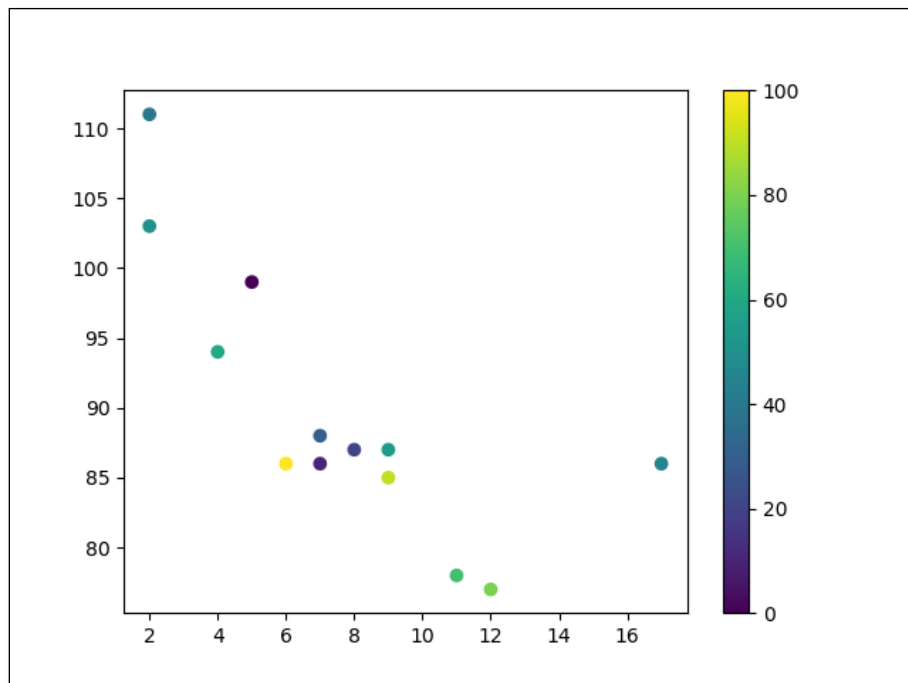
```
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
```

```
colors = np.array([0, 10, 20, 30, 40, 45, 50, 55, 60, 70, 80, 90, 100])
```

```
plt.scatter(x, y, c=colors, cmap='viridis')
```

```
plt.colorbar()
```

```
plt.show()
```



Available ColorMaps

You can choose any of the built-in colormaps:

Name	Reverse
<i>Accent</i>	<i>Accent_r</i>
<i>Blues</i>	<i>Blues_r</i>
<i>BrBG</i>	<i>BrBG_r</i>
<i>BuGn</i>	<i>BuGn_r</i>
<i>BuPu</i>	<i>BuPu_r</i>
<i>CMRmap</i>	<i>CMRmap_r</i>
<i>Dark2</i>	<i>Dark2_r</i>
<i>GnBu</i>	<i>GnBu_r</i>
<i>Greens</i>	<i>Greens_r</i>
<i>Greys</i>	<i>Greys_r</i>
<i>OrRd</i>	<i>OrRd_r</i>
<i>Oranges</i>	<i>Oranges_r</i>
<i>PRGn</i>	<i>PRGn_r</i>
<i>Paired</i>	<i>Paired_r</i>
<i>Pastel1</i>	<i>Pastel1_r</i>
<i>Pastel2</i>	<i>Pastel2_r</i>
<i>PiYG</i>	<i>PiYG_r</i>
<i>PuBu</i>	<i>PuBu_r</i>
<i>PuBuGn</i>	<i>PuBuGn_r</i>
<i>PuOr</i>	<i>PuOr_r</i>
<i>PuRd</i>	<i>PuRd_r</i>
<i>Purples</i>	<i>Purples_r</i>

<i>RdBu</i>	<i>RdBu_r</i>
<i>RdGy</i>	<i>RdGy_r</i>
<i>RdPu</i>	<i>RdPu_r</i>
<i>RdYlBu</i>	<i>RdYlBu_r</i>
<i>RdYlGn</i>	<i>RdYlGn_r</i>
<i>Reds</i>	<i>Reds_r</i>
<i>Set1</i>	<i>Set1_r</i>
<i>Set2</i>	<i>Set2_r</i>
<i>Set3</i>	<i>Set3_r</i>
<i>Spectral</i>	<i>Spectral_r</i>
<i>Wistia</i>	<i>Wistia_r</i>
<i>YlGn</i>	<i>YlGn_r</i>
<i>YlGnBu</i>	<i>YlGnBu_r</i>
<i>YlOrBr</i>	<i>YlOrBr_r</i>
<i>YlOrRd</i>	<i>YlOrRd_r</i>
<i>afmhot</i>	<i>afmhot_r</i>
<i>autumn</i>	<i>autumn_r</i>
<i>binary</i>	<i>binary_r</i>
<i>bone</i>	<i>bone_r</i>
<i>brg</i>	<i>brg_r</i>
<i>bwr</i>	<i>bwr_r</i>
<i>cividis</i>	<i>cividis_r</i>
<i>cool</i>	<i>cool_r</i>
<i>coolwarm</i>	<i>coolwarm_r</i>
<i>copper</i>	<i>copper_r</i>
<i>cubehelix</i>	<i>cubehelix_r</i>

<i>flag</i>	<i>flag_r</i>
<i>gist_earth</i>	<i>gist_earth_r</i>
<i>gist_gray</i>	<i>gist_gray_r</i>
<i>gist_heat</i>	<i>gist_heat_r</i>
<i>gist_ncar</i>	<i>gist_ncar_r</i>
<i>gist_rainbow</i>	<i>gist_rainbow_r</i>
<i>gist_stern</i>	<i>gist_stern_r</i>
<i>gist_yarg</i>	<i>gist_yarg_r</i>
<i>gnuplot</i>	<i>gnuplot_r</i>
<i>gnuplot2</i>	<i>gnuplot2_r</i>
<i>gray</i>	<i>gray_r</i>
<i>hot</i>	<i>hot_r</i>
<i>hsv</i>	<i>hsv_r</i>
<i>inferno</i>	<i>inferno_r</i>
<i>jet</i>	<i>jet_r</i>
<i>magma</i>	<i>magma_r</i>
<i>nipy_spectral</i>	<i>nipy_spectral_r</i>
<i>ocean</i>	<i>ocean_r</i>
<i>pink</i>	<i>pink_r</i>
<i>plasma</i>	<i>plasma_r</i>
<i>prism</i>	<i>prism_r</i>
<i>rainbow</i>	<i>rainbow_r</i>
<i>seismic</i>	<i>seismic_r</i>
<i>spring</i>	<i>spring_r</i>
<i>summer</i>	<i>summer_r</i>
<i>tab10</i>	<i>tab10_r</i>

<code>tab20</code>	<code>tab20_r</code>
<code>tab20b</code>	<code>tab20b_r</code>
<code>tab20c</code>	<code>tab20c_r</code>
<code>terrain</code>	<code>terrain_r</code>
<code>twilight</code>	<code>twilight_r</code>
<code>twilight_shifted</code>	<code>twilight_shifted_r</code>
<code>viridis</code>	<code>viridis_r</code>
<code>winter</code>	<code>winter_r</code>

Size

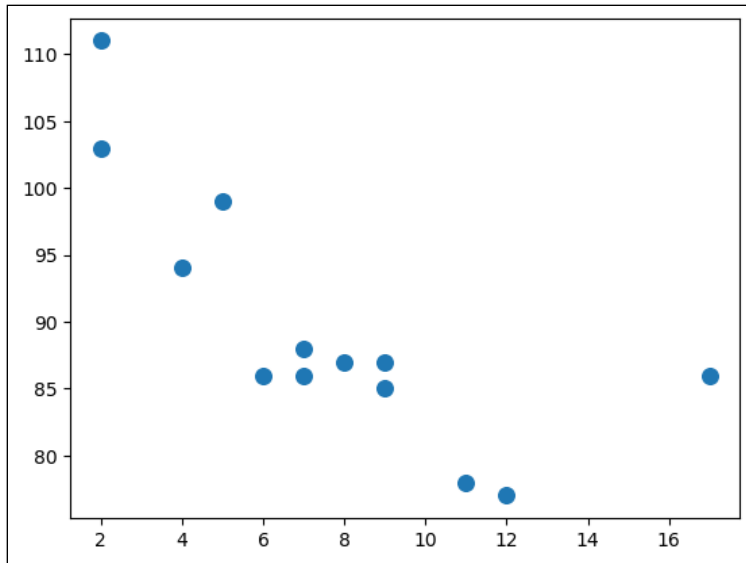
You can change the size of the dots with the **s** argument.

Just like colors, make sure the array for sizes has the same length as the arrays for the x- and y-axis:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])

plt.scatter(x, y, s=70)
plt.show()
```



Alpha

You can adjust the transparency of the dots with the alpha argument.

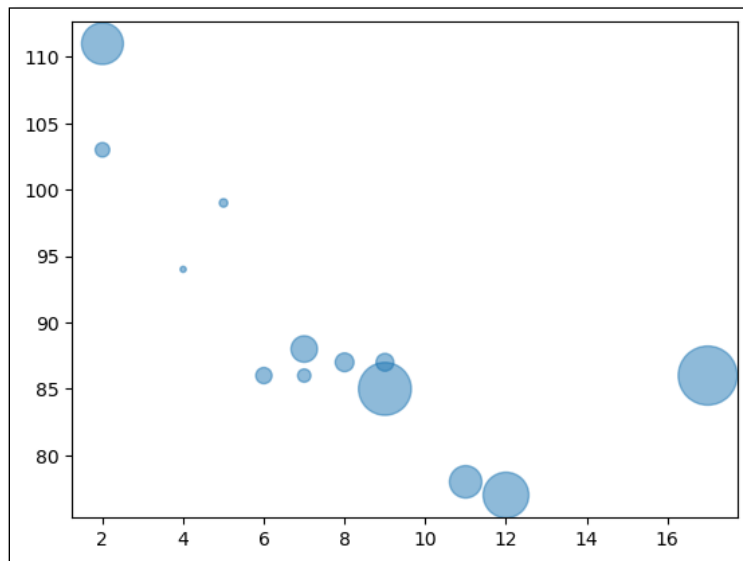
Just like colors, make sure the array for sizes has the same length as the arrays for the x- and y-axis:

Set your own size for the markers:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])
y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])
sizes=np.array([20,50,100,200,500,1000,60,90,10,300,600,800,75])
```

```
plt.scatter(x, y, s=sizes, alpha=0.5)
plt.show()
```



Creating Bar plots

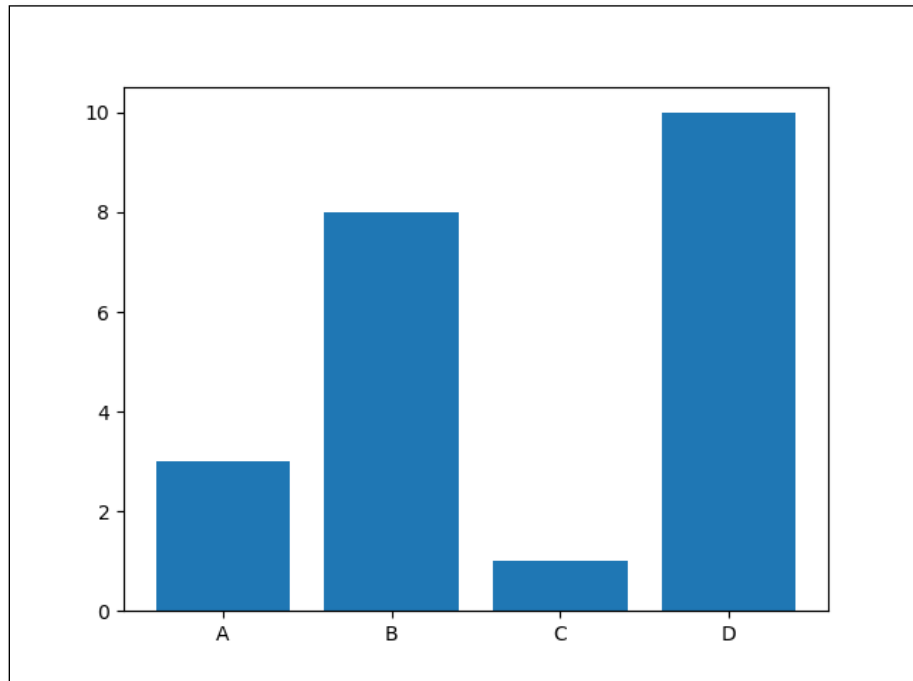
With Pyplot, you can use the **bar()** function to draw bar graphs:

Draw 4 bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y)
plt.show()
```



Horizontal Bars

If you want the bars to be displayed horizontally instead of vertically, use the `barh()` function:

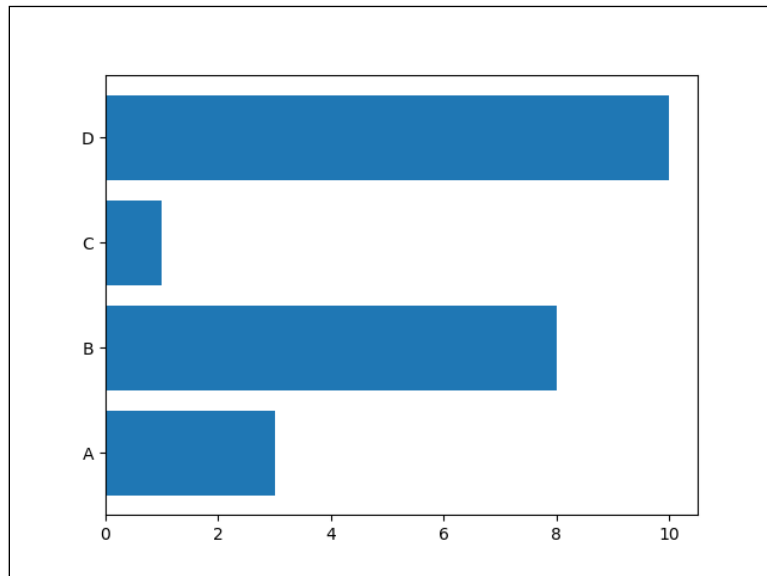
Draw 4 horizontal bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.barh(x, y)
```

```
plt.show()
```



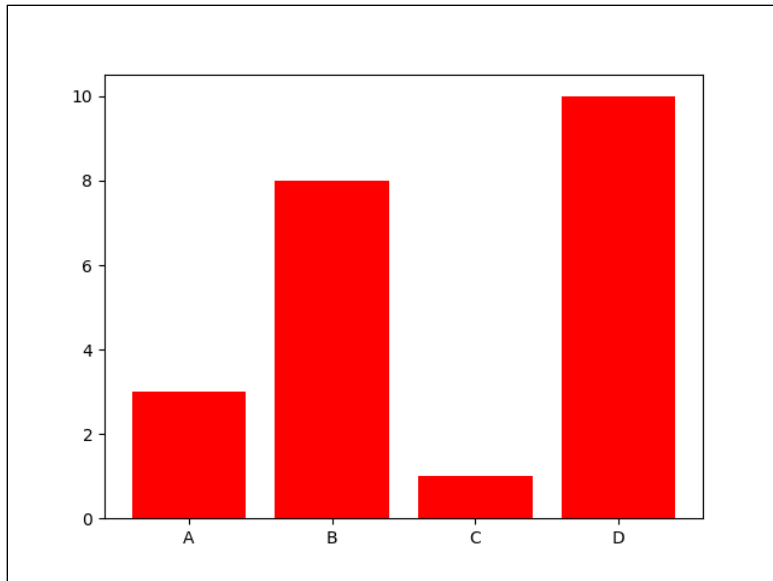
Bar Color

The `bar()` and `barh()` take the keyword argument `color` to set the color of the bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, color = "red")
plt.show()
```



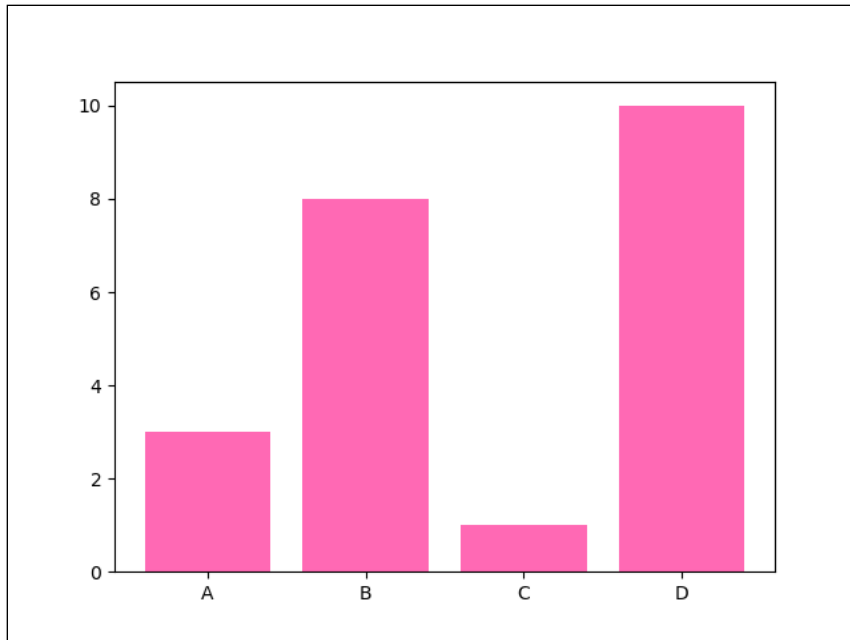
Color Names

You can use any of the 140 supported color names.

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x, y, color = "hotpink")
plt.show()
```

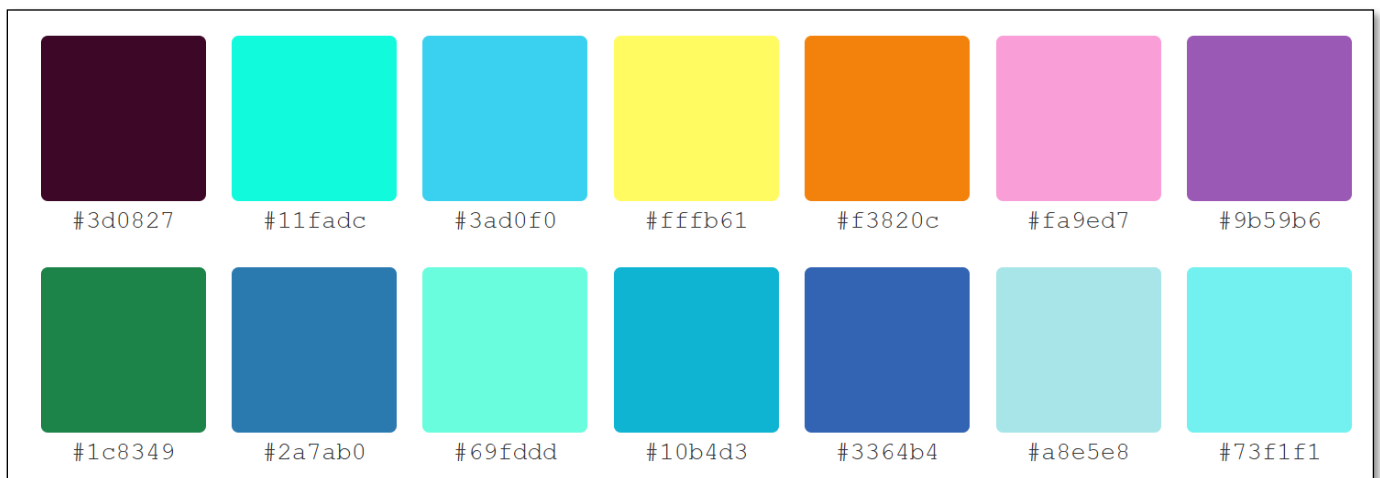


Color Hex

A **hex color code** is a six-digit combination of numbers and letters, preceded by a # symbol. Each pair of characters in the code represents the intensity of red, green, and blue (RGB) in a particular color.

The format is as follows:

#RRGGBB

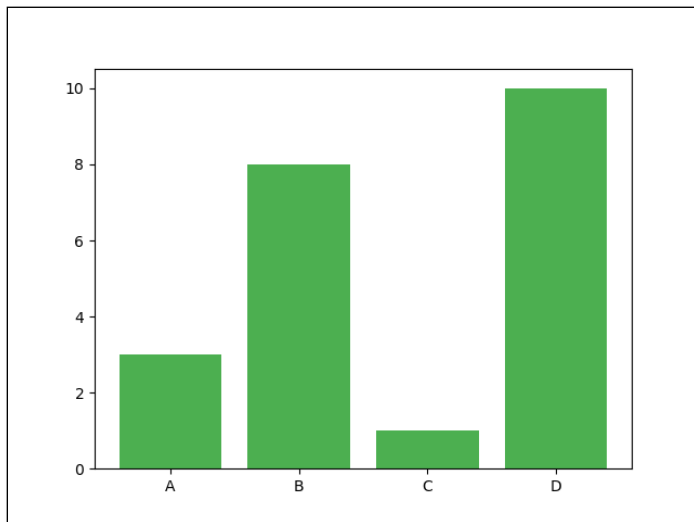


you can use Hexadecimal color values:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
```

```
plt.bar(x, y, color = "#4CAF50")
plt.show()
```



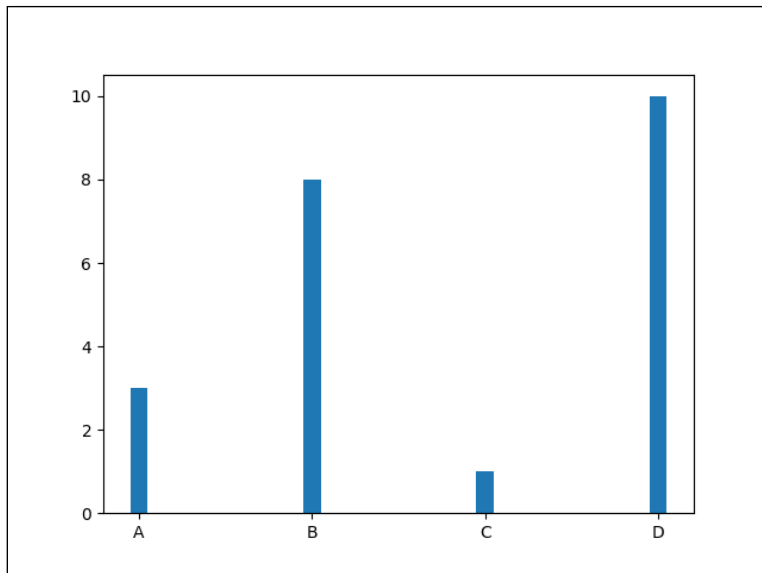
Bar Width

The bar() takes the keyword argument width to set the width of the bars:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
```

```
plt.bar(x, y, width = 0.1)
plt.show()
```



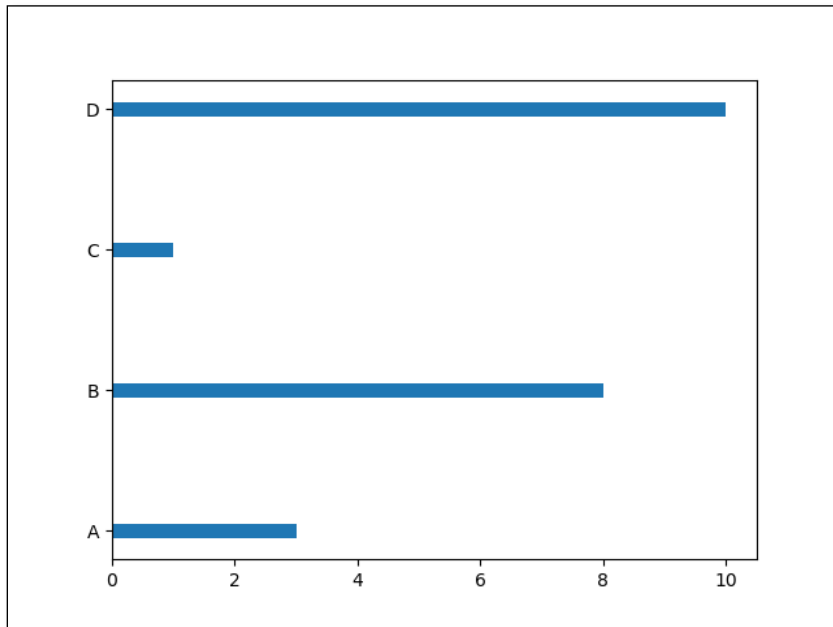
Bar Height

The `barh()` takes the keyword argument `height` to set the height of the bars:

```
import matplotlib.pyplot as plt
import numpy as np

x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.barh(x, y, height = 0.1)
plt.show()
```



Bar Chart with Custom Colors and Axis Labels

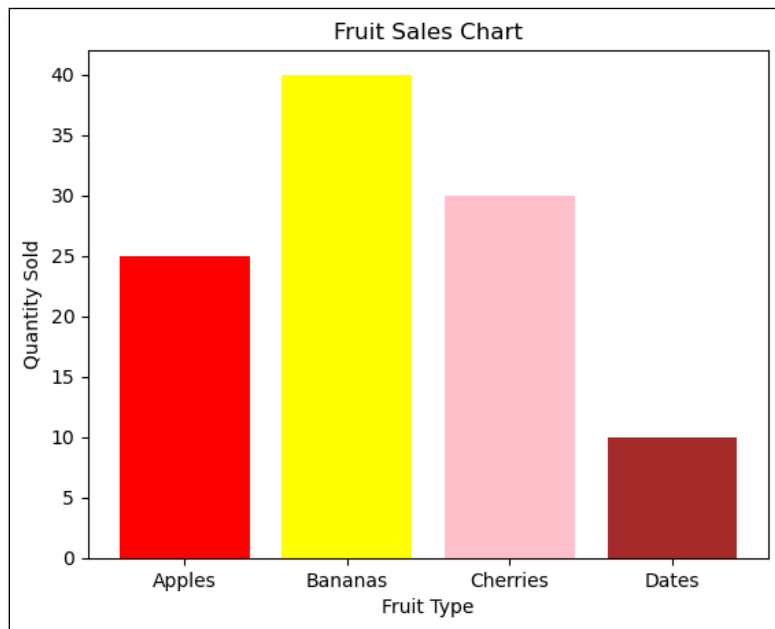
```
import matplotlib.pyplot as plt

categories = ['Apples', 'Bananas', 'Cherries', 'Dates']
values = [25, 40, 30, 10]
colors = ['red', 'yellow', 'pink', 'brown']

plt.bar(categories, values, color=colors)

plt.xlabel('Fruit Type')
plt.ylabel('Quantity Sold')
plt.title('Fruit Sales Chart')
```

```
plt.show()
```



Bar chart with labels on each bar

```
import matplotlib.pyplot as plt
```

```
years = [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]
```

```
mpg = [17.7, 21.2, 18.7, 17.1, 22.7, 20.3, 21.6, 23.4]
```

```
plt.bar(years, mpg)
```

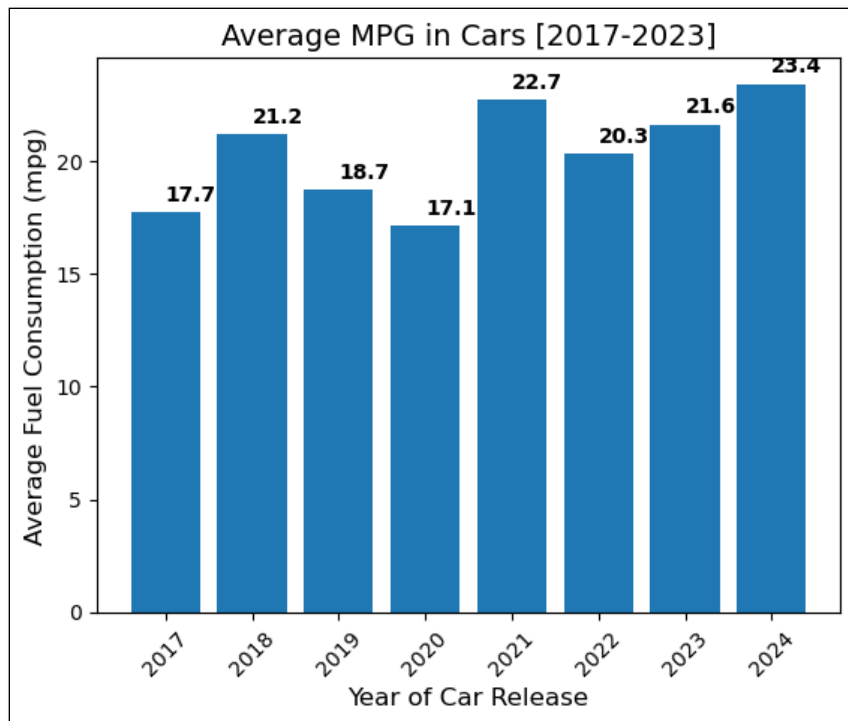
```
for i in range(len(mpg)):
```

```
    plt.text(years[i], mpg[i] + 0.5, str(mpg[i]),
```

```
    ha='center', fontsize=10, fontweight='bold')
```

```
plt.xlabel('Year of Car Release', fontsize=12)
plt.ylabel('Average Miles per Gallon (mpg)', fontsize=12)
plt.title('Average MPG in Cars [2017-2023]', fontsize=14)

plt.xticks(years, rotation=45)
plt.show()
```



Stacked and grouped plots

A **stacked bar chart** is a type of bar graph that shows how different categories contribute to a total value. Each bar is divided into segments, where each segment represents a portion of the total.

```

import matplotlib.pyplot as plt
import numpy as np

months = ['Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep']
desktop = [10000, 11000, 12000, 16000, 11000, 10500]
mobile = [8000, 7500, 9000, 9500, 8800, 8700]

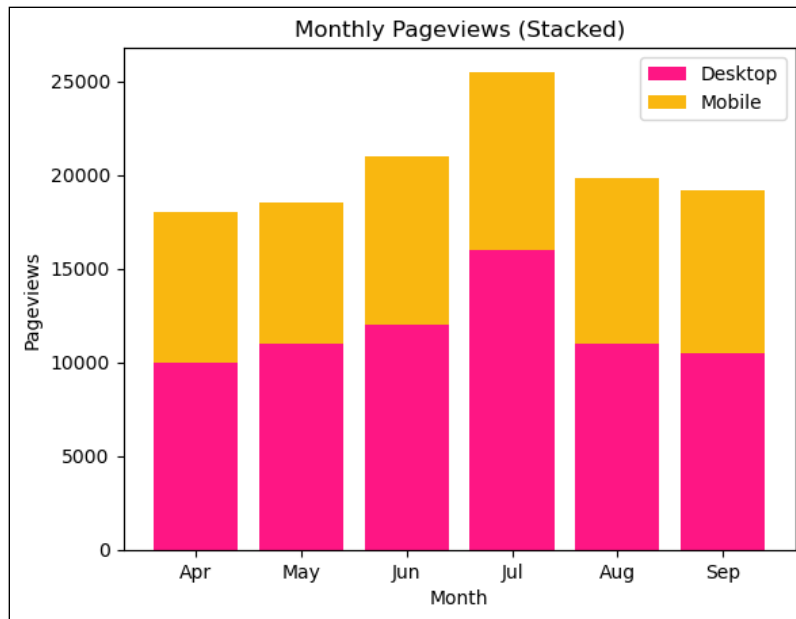
x = np.arange(len(months)) # X-axis positions

plt.bar(x, desktop, label='Desktop', color='#fe1684')
plt.bar(x, mobile, bottom=desktop, label='Mobile',
color='#f9b710')

plt.xticks(x, months)
plt.xlabel('Month')
plt.ylabel('Pageviews')
plt.title('Monthly Pageviews (Stacked)')
plt.legend()

plt.show()

```



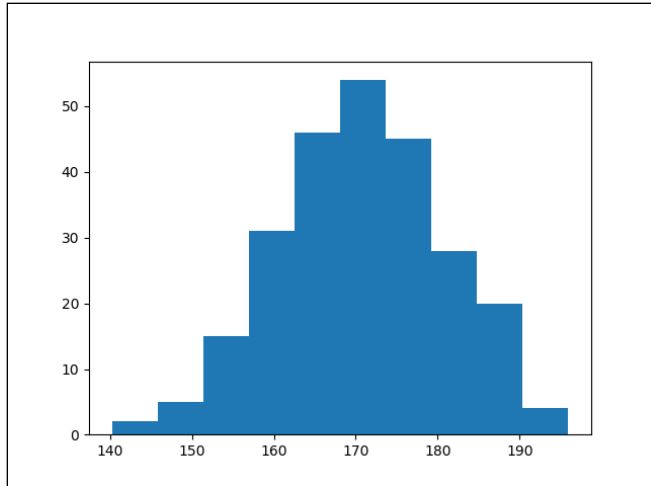
bottom=desktop means desktop value 0 to 10000 0 to 11000 etc are plotted at bottom and the mobile value will be 10000+8000 11000 to 18500

- *The **desktop** bars will go from 0 to [10000, 11000, 12000, 16000, 11000, 10500].*
- *The **mobile** bars will start at those heights, and go from [10000→ 18000], [11000→ 18500], and [12000→ 21000...] respectively.*

Histogram

A histogram is a graph showing *frequency* distributions. It is a graph showing the number of observations within each given interval.

Example: Say you ask for the height of 250 people, you might end up with a histogram like this



You can read from the histogram that there are approximately:

2 people from 140 to 145cm

5 people from 145 to 150cm

15 people from 151 to 156cm

31 people from 157 to 162cm

46 people from 163 to 168cm

53 people from 168 to 173cm

45 people from 173 to 178cm

28 people from 179 to 184cm

21 people from 185 to 190cm

4 people from 190 to 195cm

Create Histogram

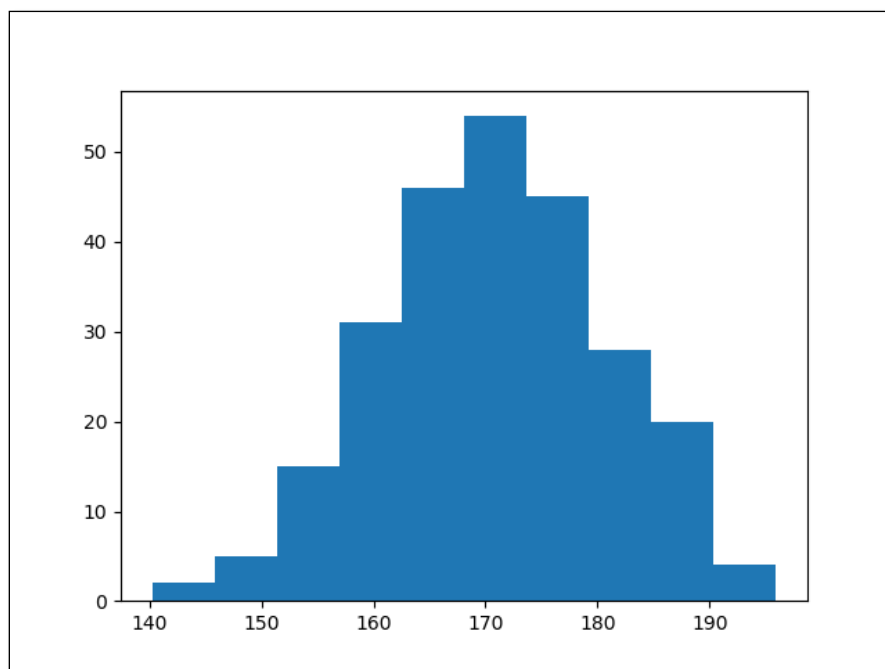
In Matplotlib, we use the `hist()` function to create histograms.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.random.normal(170, 10, 250)
```

```
plt.hist(x)
```

```
plt.show()
```



Creating Pie Charts

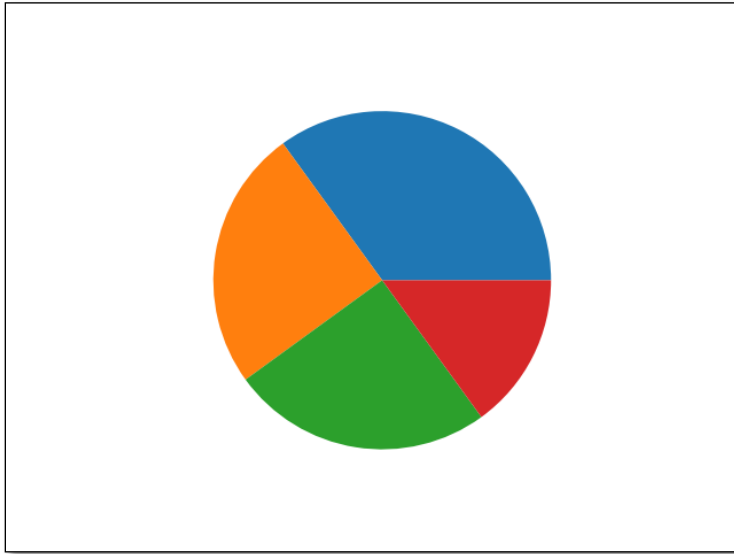
With Pyplot, you can use the `pie()` function to draw pie charts:

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
y = np.array([35, 25, 25, 15])
```

```
plt.pie(y)
plt.show()
```



Labels

Add labels to the pie chart with the labels parameter.

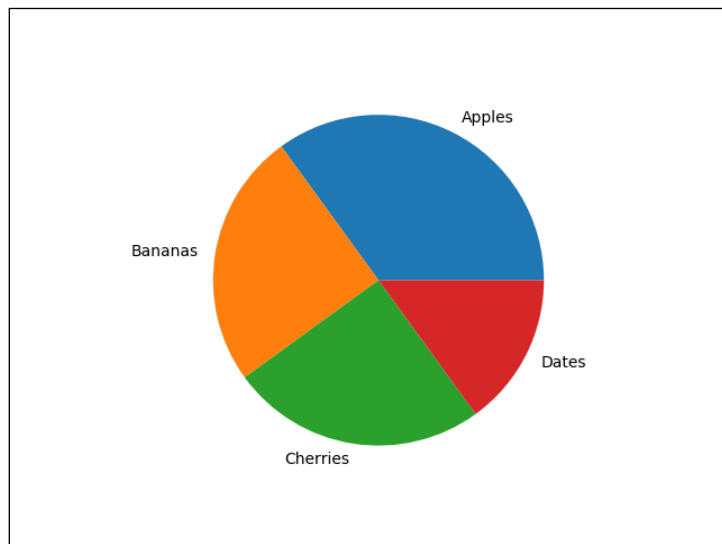
The labels parameter must be an array with one label for each wedge:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
```

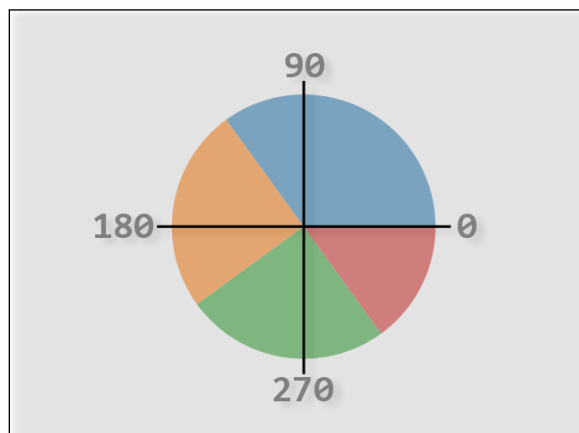
```
plt.show()
```



Start Angle

As mentioned the default start angle is at the x-axis, but you can change the start angle by specifying a startangle parameter.

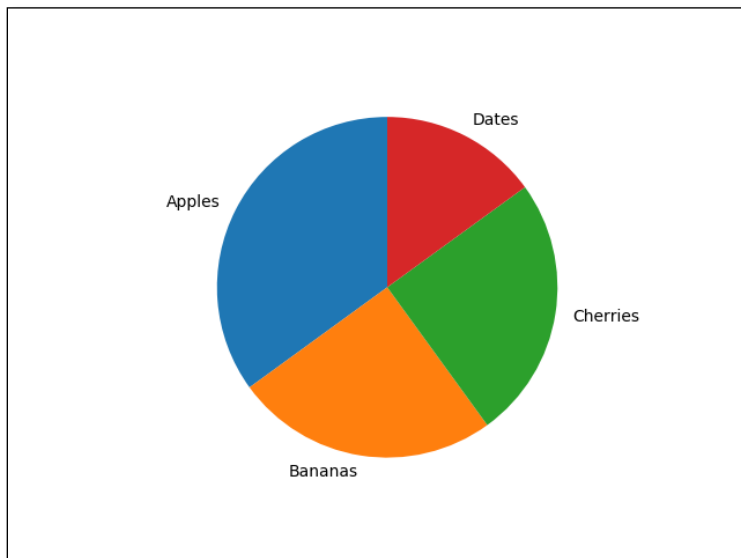
The startangle parameter is defined with an angle in degrees, default angle is 0:



```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels, startangle = 90)
plt.show()
```



Explode

Maybe you want one of the wedges to stand out? The explode parameter allows you to do that.

The explode parameter, if specified, and not None, must be an array with one value for each wedge.

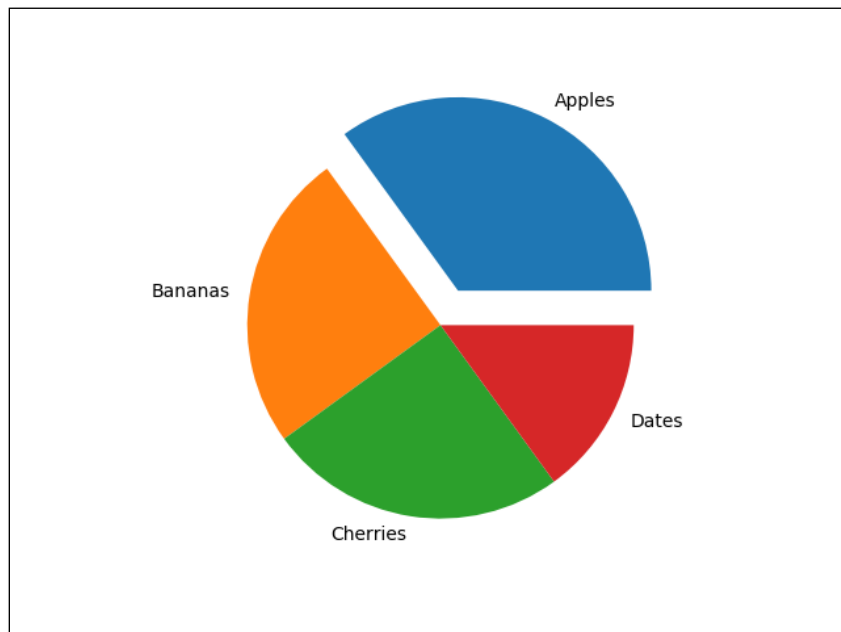
Each value represents how far from the center each wedge is displayed:

Pull the "Apples" wedge 0.2 from the center of the pie:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode)
plt.show()
```



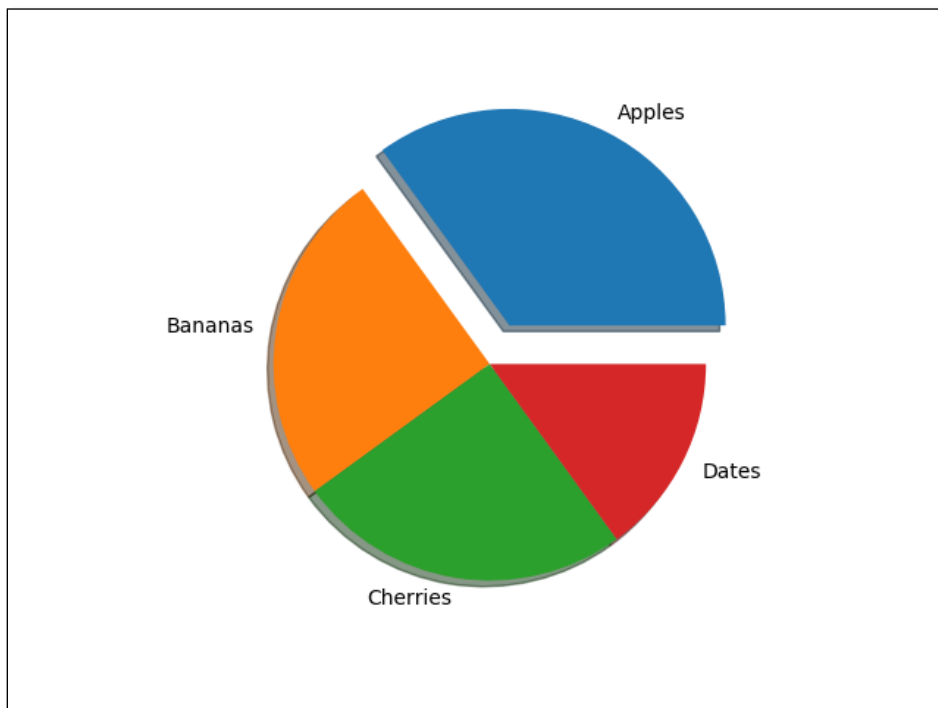
Shadow

Add a shadow to the pie chart by setting the shadows parameter to True:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode = myexplode, shadow =
True)
plt.show()
```



Colors

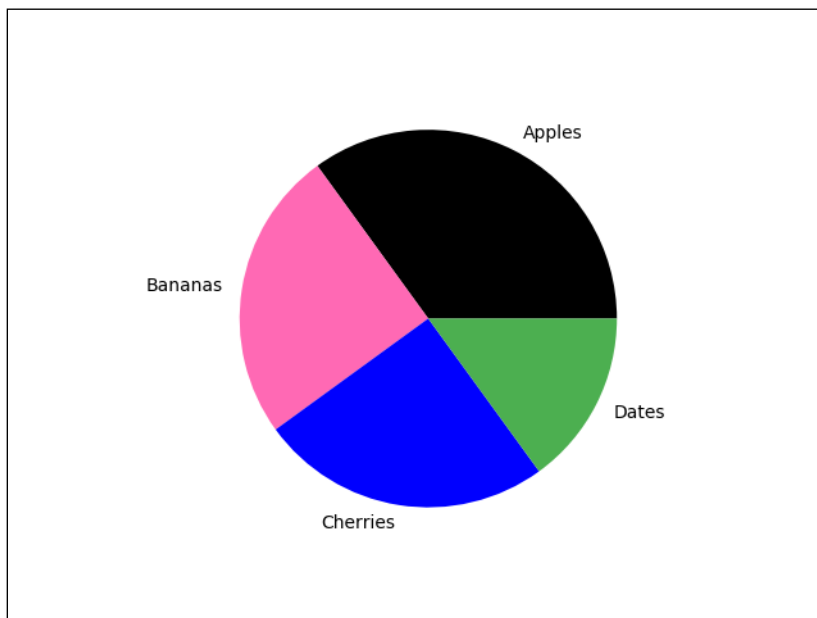
You can set the color of each wedge with the colors parameter.

The colors parameter, if specified, must be an array with one value for each wedge:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
mycolors = ["black", "hotpink", "b", "#4CAF50"]

plt.pie(y, labels = mylabels, colors = mycolors)
plt.show()
```



You can use Hexadecimal color values, any of the 140 supported color names, or one of these shortcuts:

'r' - Red
'g' - Green
'b' - Blue
'c' - Cyan
'm' - Magenta
'y' - Yellow
'k' - Black
'w' - White

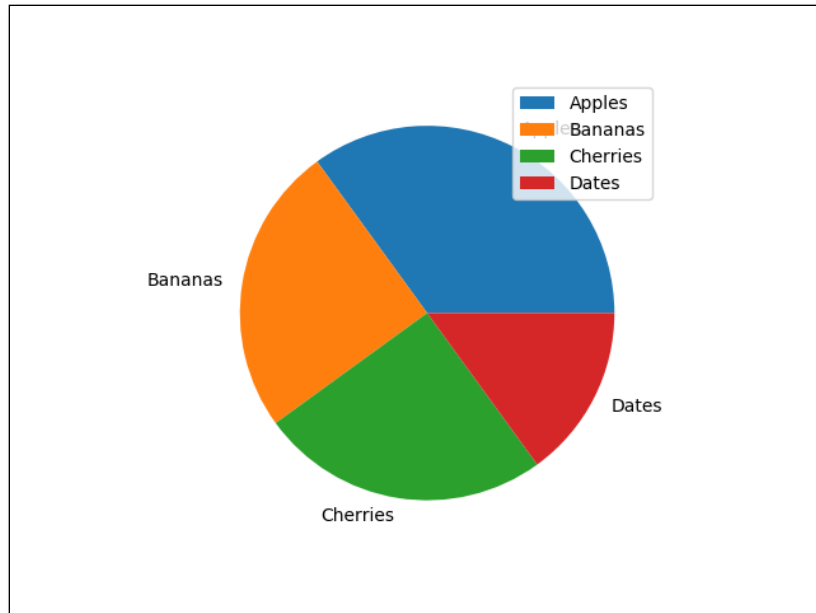
Legend

To add a list of explanation for each wedge, use the legend() function:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.legend()
plt.show()
```



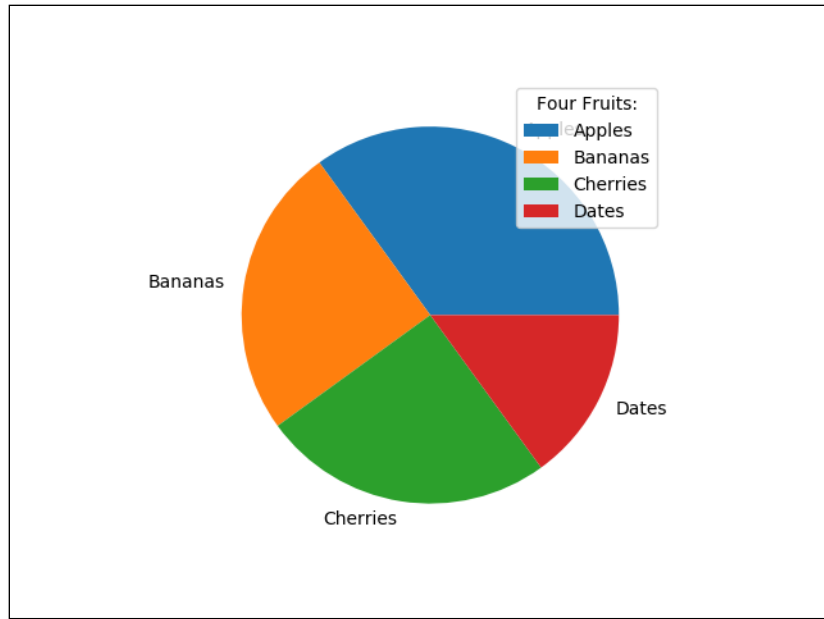
Legend With Header

To add a header to the legend, add the title parameter to the legend function.

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
mylabels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels = mylabels)
plt.legend(title = "Four Fruits:")
plt.show()
```



In a pie chart, the `autopct` parameter is used to format the display of percentage labels on the pie slices.

```
import matplotlib.pyplot as plt
import numpy as np

mylabels = ["Apples", "Bananas", "Cherries", "Dates"]
sizes = [15, 30, 45, 10]

plt.pie(sizes, labels=mylabels, autopct="%1.1f%%")
plt.show()
```

What does `"%1.1f%%"` mean?

It's a **string formatting** expression:

- `%`: indicates a format specifier is coming.
- `1.1f`:

- 1: minimum number of characters to be displayed (optional here).
 - .1: number of decimal places (1 digit after the decimal point).
 - f: floating-point number.
- %: escapes the % character so it is displayed literally.
 - If a slice represents 35% of the pie, "%1.1f%" would format it as: 35.0%
 - "%.0f%" → rounds to whole numbers, e.g., 35%
 - "%.2f%" → shows two decimal places, e.g., 35.00%

